11

Templates and Exception Handling

Chapter Objectives

11.1 Generic functions

11.2 Generic classes

11.3 Exception handling

11.4 More about exception handling

11.5 Handling exceptions thrown by **new**

THIS chapter discusses two of C++'s most important high-level features: *templates* and *exception handling*. While neither was part of the original specification for C++, both were added several years ago and are defined by Standard C++. They are supported by all modern C++ compilers. These two features help you achieve two of the most elusive goals in programming: the creation of reusable and resilient code.

Using templates, it is possible to create generic functions and classes. In a generic function or class, they type of data that operated upon a specified as a parameter. This allows you to use one function or class with several different types of data without having to explicitly recode a specific version for each different data type. Thus, templates allow you to create reusable code. Both generic functions and generic classes are discussed here.

Exception handling is the subsystem of C++ that allows you to handle errors that occur at run time in a structured and controlled manner. With C++ exception handling, your program can automatically invoke an error handling routine when an error occurs. The principal advantage of exception handling is that it automates much of the error handling code that previously had to be coded "by hand" in any large program. The proper use of exception handling helps you to create resilient code.

Review Skills Check

Before proceeding, you should be able to correctly answer the following questions and do the exercises.

1. What is a virtual function?
2. What is a pure virtual function? If a class declaration contains a pure virtual function, what is that class called, and what restrictions apply to its usage?
3. Run-time polymorphism is achieved through the use of _____ functions and _____ class pointers. (Fill in the missing words.)
4. If, in a class hierarchy, a derived class neglects to override a (non-pure) virtual function, what happens when an object of that derived class calls that function?
5. What is the main advantage of run-time polymorphism? What is its potential disadvantage?

11.1 GENERIC FUNCTIONS

A generic function defines a general set of operations that will be applied to various types of data. A generic function has the type of data that it will operate upon passed to it as a parameter. Using this mechanism, the same general procedure can be applied to a wide range of data. As you know, many algorithms are logically the same no matter what type of data is being operated upon. For example, the Quicksort algorithm is the same whether it is applied to an array of integers or an array of **floats**. It is just that the type of the data being sorted is different. By creating a generic function, you can define, independent of any data, the nature of the algorithm. Once this is done, the compiler automatically generates the correct code for the type of data that is actually used when you execute the function. In essence, when you create a generic function you are creating a function that can automatically overload itself.

A generic function is created using the keyword **template**. The normal meaning of the word *template* accurately reflects the keyword's use in C++. It is used to create a template (or framework) that describes what a function will do, leaving it to the compiler to fill in the details as needed. The general form of a template function definition is shown here:

```
template<class Ttype>ret_type func_name(parameter list)
{
    // body of function
}
```

Here *Ttype* is a placeholder name for a data type used by the function. This name can be used within the function definition. However, it is only a placeholder; the compiler will automatically replace this placeholder with an actual data type when it creates a specific version of the function.
Although the use of the keyword **class** to specify a generic type in a template declaration is traditional, you can also use the keyword **typename**.

EXAMPLES

1. The following program creates a generic function that swaps the values of the two variables it is called with. Because the general process of exchanging two values is independent of the type of the variables, this process is a good choice to be made into a generic function.

```
// Function template example.
#include <iostream>
using namespace std;

// This is a function template.
template <class X> void swapargs(X &a, X &b)
{
    X temp;

    temp = a;
    a = b;
    b= temp;
}

int main()
{
    int i=10, j=20;
    float x=10, y=23.3;

    cout << "Original i, j: " << i << ' '  << j << endl;
    cout << "Original x, y: " << x << ' '  << y << endl;

    swapargs(i, j); // swap integers
    swapargs(x, y); // swap floats

    cout << "Swapped i, j: " << i << ' '  << j << endl;
    cout << "Swapped x, y: " << x << ' '  << y << endl;

    return 0;
}
```

The keyword **template** is used to define a generic function. The line:

```
template <class X> void swapargs(X &a, X &b)
```

tells the compiler two things: that a template is being created and that a generic definition is beginning. Here **X** is a generic type that is used as a placeholder. After the **template** portion, the function **swapargs()** is declared, using **X** as the data type of the values that will be swapped. In **main()**, the **swapargs()** function is called using two different types of data: integers and **float**s. Because **swapargs()** is a generic function, the compiler automatically creates two versions of **swapargs()**-one that will exchange integer values and one that will exchange floating-point values. You should compile and try this program now.

Here are some other terms that are sometimes used when templates are discussed and that you might encounter in other C++ literature. First, a generic function (that is, a function definition preceded by a **template** statement) is also called a *template function.* When the compiler creates a specific version of this function, it is said to have created a *generated function.* The act of generating a function is referred to as *instantiating* it. Put differently, a generated function is a specific instance of a template function.

2. The **template** portion of a generic function definition does not have to be on the same line as the function's name. For example, the following is also a common way to format the **swapargs()** function:

```
template <class X>
void swapargs(X &a, X &b)
{
     X temp;

     temp = a;
     a = b;
     b= temp;
}
```

If you use this form, it is important to understand that no other statements can occur between the **template** statement and the start of the generic function definition. For example, the following fragment will not compile:

```
// This will not compile.
template <class X>
int i; // this is an error
void swapargs(X &a, X &b)
{
     X temp;

     temp = a;
     a = b;
     b= temp;
}
```

As the comments imply, the **template** specification must directly precede the rest of the function definition.

3. As mentioned, instead of using the keyword **class**, you can use the keyword **typename** to specify a generic type in a template definition. For example, here is another way to declare the **swapargs()** function.

```
// Use typename
template <typename X> void swapargs(X &a, X &b)
```

```
{
     X temp;

     temp = a;
     a = b;
     b= temp;
}
```

The **typename** keyword can also be used to specify an unknown type within a template, but this use is beyond the scope of this book.

4. You can define more than one generic data type with the **template** statement, using a comma-separated list. For example, this program creates a generic function that has two generic types:

```
#include <iostream>
using namespace std;

template <class type1, class type2>
void myfunc(type1 x, type2 y)
{
     cout << x << ' ' << y << endl;
}

int main()
{
     myfunc(10, "hi");

     myfunc(0.23, 10L);

     return 0;
}
```

In this example, the placeholder types **type1** and **type2** are replaced by the compiler with the data types **int** and **char *** and **double** and **long**, respectively, when the compiler generates the specific instances of **myfunc()**.

Remember: *When you create a generic function, you are, in essence, allowing the compiler to generate as many different versions of that function as necessary to handle the various ways that your program calls that function.*

5. Generic functions are similar to overloaded functions except that they are more restrictive. When functions are overloaded, you can have different actions performed within the body of each function. But a generic function must perform the same general action for all versions. For example, the following overloaded functions *cannot* be replaced by a generic function because they do not do the same thing:

```
void outdata(int i)
{
     cout << i;
}

void outdata(double d)
{
     cout << setprecision(10) << setfill('#');
```

```
    cout << d;
    cout << setprecision(6) << setfill(' ');
}
```

6. Even though a template function overloads itself as needed, you can explicitly overload one, too. If you overload a generic function, that overloaded function overrides (or *hides*) the generic function relative to that specific version. For example, consider this version of Example 1:

```
// Overriding a template function.
#include <iostream>
using namespace std;

// This is a function template.
template <class X> void swapargs(X &a, X &b)
{
    X temp;

    temp = a;
    a = b;
    b= temp;
}

// This overrides the generic version of swapargs().
void swapargs(int a, int b)
{
    cout << "this is inside swapargs(int,int)\n";
}

int main()
{
    int i=10, j=20;
    float x=10, y=23.3;

    cout << "Original i, j: " << i << ' '  << j << endl;
    cout << "Original x, y: " << x << ' '  << y << endl;

    swapargs(i, j); // calls overloaded swapargs()
    swapargs(x, y); // swap floats

    cout << "Swapped i, j: " << i << ' '  << j << endl;
    cout << "Swapped x, y: " << x << ' '  << y << endl;

    return 0;
}
```

As the comments indicate, when **swapargs(i,j)** is called, it invokes the explicitly overloaded version of **swapargs()** defined in the program. Thus, the compiler does not generate this version of the generic **swapargs()** function because the generic function is overridden by the explicit overloading.

Manual overloading of a template, as shown in this example, allows you to tailor a version of a generic function to accommodate a special situation. However, in general, if you need

to have different versions of a function for different data types, you should use overloaded functions rather than templates.

EXERCISES

1. If you have not done so, try each of the preceding examples.
2. Write a generic function, called **min()**, that returns the lesser of its two arguments. For example, **min(3, 4)** will return 3 and **min('c', 'a'** will return a. Demonstrate your function in a program.
3. A good candidate for a template function is called **find()**. This function searches an array for an object. It returns either the index of the matching object (if one is found) or -1 if no match is found. Here is the prototype for a specific version of **find()**. Convert **find()** into a generic function and demonstrate your solution within a program. (The **size** parameter specifies the number of elements in the array.)

```
int find(int object, int *list, int size)
{
    // ...
}
```

4. In your own words, explain why generic functions are valuable and may help simplify the source code to program that you create.

11.2 GENERIC CLASSES

In addition to defining generic functions, you can also define generic classes. When you do this, you create a class that defines all algorithms used by that class, but the actual type of the data being manipulated will be specified as a parameter when objects of that class are created. Generic classes are useful when a class contains generalizable logic. For example, the same algorithm that maintains a queue of integers will also work for a queue of characters. Also, the same mechanism that maintains a linked list of mailing addresses will also maintain a linked list of auto part information. By using a generic class, you can create a class that will maintain a queue, a linked list, and so on for any type of data. The compiler will automatically generate the correct type of object based upon the type you specify when the object is created.
The general form of a generic class declaration is shown here:

```
template <class Ttype> class class_name
{
    .
    .
    .
};
```

Here *Ttype* is the placeholder type name that will be specified when a class is instantiated. If necessary, you can define more than one generic data type by using a comma-separated list.
Once you have created a generic class, you create a specific instance of that class by using the following general form:

```
class_name <type> ob;
```

Here *type* is the type name of the data that the class will be operating upon.
Member functions of a generic class are, themselves, automatically generic. They need not be explicitly specified as such using **template**.

As you will see in Chapter 14, C++ provides a library that is built upon template classes. This library is usually referred to as the Standard Template Library, or STL for short. It provides generic versions of the most commonly used algorithms and data structures. If you want to use the STL effectively, you'll need a solid understanding of template classes and their syntax.

EXAMPLES

1. This program creates a very simple generic singly linked list class. It then demonstrates the class by creating a linked list that stores characters.

```
// A simple generic linked list.
#include <iostream>
using namespace std;

template <class data_t> class list
{
    data_t data;
    list *next;
public:
    list(data_t d);
    void add(list *node) { node->next = this; next = 0; }
    list *getnext() { return next; }
    data_t getdata() { return data; }
};

template <class data_t> list<data_t>::list(data_t d)
{
    data = d;
    next = 0;
}

int main()
{
    list<char> start('a');
    list<char> *p, *last;
    int i;

    // build a list
    last = &start;
    for(i=1; i<26; i++)
    {
        p = new list<char>('a' + i);
        p->add(last);
        last = p;
    }

    // follow the list
    p = &start;
    while(p)
    {
        cout << p->getdata();
        p = p->getnext();
```

```
    }

    return 0;
}
```

As you can see, the declaration of a generic class is similar to that of a generic function. The actual type of data stored by the list is generic in the class declaration. It is not until an object of the list is declared that the actual data type is determined. In this example, objects and pointers are created inside **main()** that specify that the data type of the list will be **char**. Pay special attention to this declaration:

```
list<char> start('a');
```

Notice how the desired data type is passed inside the angle brackets.

You should enter and execute this program. It builds a linked list that contains the characters of the alphabet and then displays them. However, by simply changing the type of data specified when **list** objects are created, you can change the type of data stored by the list. For example, you could create another object that stores integers by using this declaration:

```
list<int> int_start(1);
```

You can also use **list** to store data types that you create. For example, if you want to store address information, use this structure:

```
struct addr
{
    char name[40];
    char street[40];
    char city[30];
    char state[3];
    char zip[12];
}
```

Then, to use **list** to generate objects that will store objects of type **addr**, use a declaration like this (assuming that **structvar** contains a valid **addr** structure):

```
list<addr> obj(structvar);
```

2. Here is another example of a generic class. It is a reworking of the **stack** class first introduced in Chapter 1. However, in this case, **stack** has been made into a template class. Thus, it can be used to store any type of object. In the example, shown here, a character stack and a floating-point stack are created:

```
// This function demonstrates a generic stack.
#include <iostream>
using namespace std;

#define SIZE 10

// Create a generic stack class
template <class StackType> class stack
{
    StackType stck[SIZE]; // holds the stack
    int tos; // index of top of stack
```

```
public:
    void init() { tos = 0; } // initialize stack
    void push(StackType ch); // push object on stack
    StackType pop(); // pop object from stack
};

// Push an object.
template <class StackType>
void stack<StackType>::push(StackType ob)
{
    if(tos==SIZE)
    {
        cout << "Stack is full.\n";
        return;
    }
    stck[tos] = ob;
    tos++;
}

// Pop an object
template <class StackType>
StackType stack<StackType>::pop()
{
    if(tos==0)
    {
        cout << "Stack is empty.\n";
        return 0; // return null on empty stack
    }
    tos--;
    return stck[tos];
}

int main()
{
    // Demonstrate character stacks.
    stack<char> s1, s2; // create two stacks
    int i;

    // initialize the stacks
    s1.init();
    s2.init();

    s1.push('a');
    s2.push('x');
    s1.push('b');
    s2.push('y');
    s1.push('c');
    s2.push('z');

    for(i=0; i<3; i++)
        cout << "Pop s1: " << s1.pop() << "\n";
```

```
    for(i=0; i<3; i++)
        cout << "Pop s2: " << s2.pop() << "\n";

    // demonstrate double stacks
    stack<double> ds1, ds2; // create two stacks

    // initialize the stacks
    ds1.init();
    ds2.init();

    ds1.push(1.1);
    ds2.push(2.2);
    ds1.push(3.3);
    ds2.push(4.4);
    ds1.push(5.5);
    ds2.push(6.6);

    for(i=0; i<3; i++)
        cout << "Pop ds1: " << ds1.pop() << "\n";
    for(i=0; i<3; i++)
        cout << "Pop ds2: " << ds2.pop() << "\n";

    return 0;
}
```

As the **stacl** class (and the preceding **list** class) illustrates, generic function and classes provide a powerful tool that you can use to maximize your programming time because they allow you to define the general form of an algorithm that can be used with any type of data. You are saved from the tedium of creating separate implementations for each data type that you want the algorithm to work with.

3. A template class can have more than one generic data type. Simply declare all the data types required by the class in a comma-separated list within the **template** specification. For example, the following short example creates a class that uses two generic data types:

```
/*
   This example uses two generic data types in a
   class definition
*/
#include <iostream>
using namespace std;

template <class Type1, class Type2> class myclass
{
    Type1 i;
    Type2 j;
public:
    myclass(Type1 a, Type2 b) { i = a; j = b; }
    void show() { cout << i << ' ' << j << '\n'; }
};

int main()
{
```

```
    myclass<int, double> ob1(10, 0.23);
    myclass<char, char *> ob2('X', "This is a test");

    ob1.show(); // show int, double
    ob2.show(); // show char, char *

    return 0;
}
```

This program produces the following output:

```
10 0.23
X This is a test
```

The program declares two types of objects. **ob1** uses integer and **double** data. **ob2** uses a character and a character pointer. For both cases, the compiler automatically generates the appropriate data and functions to accommodate the way the objects are created.

EXERCISES

1. If you have not yet done so, compile and run the two generic class examples. Try declaring lists and/or stacks of different data types.
2. Create and demonstrate a generic queue class.
3. Create a generic class, called **input**, that does the following when its constructor is called:
   - ➤ prompts the user for input,
   - ➤ inputs the data entered by the user, and
   - ➤ reprompts if the data is not within a predetermined range.

   Objects of type **input** should be declared like this:

```
input ob("prompt message", min_value, max_value)
```

   Here *prompt message* is the message that prompt for input. The minimum and maximum acceptable values are specified by *min-value* and *max-value*, respectively. (Note: the type of data entered by the user will be the same as the type of *min-value*and *max-value*.)

11.3 EXCEPTION HANDLING

C++ provides a built-in error handling mechanism that is called *exception handling*. Using exception handling, you can more easily manage and respond to run-time errors. C++ exception handling is built upon three keywords: **try**, **catch**, and **throw**. In the most general terms, program statements that you want to monitor for exceptions are contained in a **try** block. If an exception (i.e., an error) occurs within the **try** block, it is thrown (using **throw**. The exception is caught, using **catch**, and processed. The following elaborates upon this general description. As stated, any statement that throws an exception must have been executed from within a **try** block. (A function called from within a **try** block can also throw an exception.) Any exception must be caught by a **catch** statement that immediately follows the **try** statement that throws the exception. The general form of **try** and **catch** are shown here:

```
try
{
    // try block
}
catch(type1 arg)
{
    // catch block
}
catch(type2 arg)
{
    // catch block
}
catch(type3 arg)
{
    // catch block
}
.
.
.
catch(typeN arg)
{
    // catch block
}
```

The **try** block must contain the portion of your program that you want to monitor for errors. This can be as specific as monitoring a few statements within one function or as all-encompassing as enclosing the **main()** function code within a **try** block (which effectively causes the entire program to be monitored).
When an exception is thrown, it is caught by its corresponding **catch** statement, which processes the exception. There can be more than one **catch** statement associated with a **try**. The **catch** statement that is used is determined by the type of the exception. That is, if the data type specified by a **catch** matches that of the exception, that **catch** statement if executed (and all others are bypassed). When an exception is caught, *arg* will receive its value. If you don't need access to the exception itself, specify only *type* in the **catch** clause-*arg* is optional. Any type of data can be caught, including classes that you create. In fact, class types are frequently used as exceptions.
The general form of the **throw** statement is shown here:

```
throw exception;
```

throw must be executed either from within the **try** block proper or from any function that the code within the block calls (directly or indirectly). *exception* is the value thrown.
If you throw an exception for which there is no applicable **catch** statement, an abnormal program termination might occur. If your compiler compiles with Standard C++, throwing an unhandled exception causes the standard library function **terminate()** to be invoked. By default, **terminate()** calls **abort()** to stop your program, but you can specify your own termination handler, if you like. You will need to refer to your compiler's library reference for details.

EXAMPLES

1. Here is a simple example that shows the way C++ exception handling operates:

```
// A simple exception handling example.
```

```
#include <iostream>
using namespace std;

int main()
{
    cout << "start\n";

    try // start a try block
    {
        cout << "Inside try block\n";
        throw 10; // throw an error
        cout << "This will not execute";
    }
    catch(int i) // catch an error
    {
        cout << "Caught One! Number is: ";
        cout << i << "\n";
    }

    cout << "end";

    return 0;
}
```

This program displays the following output:

```
start
Inside try block
Caught One!  Number is:  10
end
```

Look carefully at this program. As you can see, there is a **try** block containing three statements and a **catch(int i)** statement that processes an integer exception. Within the **try** block, only two of the three statements will execute: the first **cout** statement and the **throw**. Once an exception has been thrown, control passes to the **catch** expression and the **try** block is terminated. That is, **catch** is *not* called. Rather, program execution is transferred to it. (The stack is automatically reset as needed to accomplish this.) Thus, the **cout** statement following the **throw** will never execute.

After the **catch** statement executes, program control continues with the statements following the **catch**. Often, however, a **catch** block will end with a call to **exit()**, **abort()**, or some other function that causes program termination because exception handling is frequently used to handle catastrophic errors.

2. As mentioned, the type of the exception must match the type specified in a **catch** statement. For example, in the preceding example, if you change the type in the **catch** statement to **double**, the exception will not be caught, and abnormal termination will occur. This change is shown here:

```
// This example will not work.
#include <iostream>
using namespace std;
```

```
int main()
{
    cout << "start\n";

    try // start a try block
    {
        cout << "Inside try block\n";
        throw 10; // throw an error
        cout << "This will not execute";
    }
    catch(double i) // won't work for an int exception
    {
        cout << "Caught One! Number is: ";
        cout << i << "\n";
    }

    cout << "end";

    return 0;
}
```

This program produces the following output because the integer exception will not be caught by a **double catch** statement.

```
start
```

```
Inside try block
```

```
Abnormal program termination
```

3. An exception can be thrown from a statement that is outside the **try** block as long as the statement is within a function that is called from within the **try** block. For example, this is a valid program:

```
/*
   Throwing an exception from a function outside
   the try block.
*/
#include <iostream>
using namespace std;

void Xtest(int test)
{
    cout << "Inside Xtest, test is: " << test << "\n";
    if(test)
        throw test;
}

int main()
{
    cout << "start\n";

    try // start a try block
    {
```

```
        Xtest(0);
        Xtest(1);
        Xtest(2);
    }
    catch(int i) // catch an error
    {
        cout << "Caught One! Number is: ";
        cout << i << "\n";
    }

    cout << "end";

    return 0;
}
```

This program produces the following output:

```
start
Inside try block
Inside Xtest, test is:  0
Inside Xtest, test is:  1
Caught One!  Number is:  1
end
```

4. A **try** block can be localized to a function. When this is the case, each time the function is entered, the exception handling relative to that function is reset. For example, examine this program:

```
#include <iostream>
using namespace std;

// A try/catch can be inside a function other than main().
void Xhandler(int test)
{
    try
    {
        if(test)
            throw test;
    }
    catch(int i)
    {
        cout << "Caught One! Ex, #: " << i << '\n';
    }
}

int main()
{
    cout << "start\n";

    Xhandler(1);
    Xhandler(2);
```

```
    Xhandler(0);
    Xhandler(3);

    cout << "end";

    return 0;
}
```

This program displays this output:

```
start
Caught One!  Ex.  #:  1
Caught One!  Ex.  #:  2
Caught One!  Ex.  #:  3
end
```

As you can see, three exceptions are thrown. After each exception, the function returns. When the function is called again, the exception handling is reset.

5. As stated earlier, you can have more than one **catch** associated with a **try**. In fact, it is common to do so. However, each **catch** must catch a different type of exception. For example, the following program catches both integers and strings:

```
#include <iostream>
using namespace std;

// Different types of exceptions can be caught.
void Xhandler(int test)
{
    try
    {
        if(test)
            throw test;
        else
            throw "Value is zero.";
    }
    catch(int i)
    {
        cout << "Caught One! Ex, #: " << i << '\n';
    }
    catch(const char *str)
    {
        cout << "Caught a string: ";
        cout << str << '\n';
    }
}

int main()
{
    cout << "start\n";

    Xhandler(1);
```

```
    Xhandler(2);
    Xhandler(0);
    Xhandler(3);

    cout << "end";

    return 0;
}
```

This program produces the following output:

```
start
Caught One!  Ex.  #:  1
Caught One!  Ex.  #:  2
Caught a string:  Value is zero
Caught One!  Ex.  #:  3
end
```

As you can see, each **catch** statement responds only to its own type.

In general, **catch** expressions are checked in the order in which they occur in a program. Only a matching statement is executed. All other **catch** blocks are ignored.

EXERCISES

1. By far, the best way to understand how C++ exception handling works is to play with it. Enter, compile, and run the preceding example programs. Then experiment with them, altering pieces of them and observing the results.

2. What is wrong with this fragment?

```
int main()
{
    throw 12.23;
```

3. What is wrong with this fragment?

```
try
{
    // ...
    throw 'a';
    // ...
}
catch(char *)
{
    // ...
}
```

4. What will happen if an exception is thrown for which there is no corresponding **catch** statement?

11.4 MORE ABOUT EXCEPTION HANDLING

There are several additional features and nuances to C++ exception handling that can make it easier and more convenient to use.
In some circumstances you will want an exception handler to catch all exceptions instead of just a certain type. This is easy to accomplish. Simply use this form of **catch**:

```
catch(...)
{
    // process all exceptions
}
```

Here the ellipsis matches any type of data.
You can restrict the type of exceptions that a function can throw back to its caller. Put differently, you can control what type of exceptions a function can throw outside of itself. In fact, you can also prevent a function from throwing any exceptions whatsoever. To apply these restrictions, you must add a **throw** clause to the function definition. The general form of this is shown here:

```
ret_type func_name(arg_list) throw(type_list)
{
    // ...
}
```

Here only those data types contained in the comma-separated *type-list* may be thrown by the function. Throwing any other type of expression will cause abnormal program termination. If you don't want a function to be able to throw *any* exceptions, use an empty list.
If your compiler complies with Standard C++, when a function attempts to throw a disallowed exception the standard library function **unexpected()** is called. By default, this causes the **terminate()** function to be called, which causes abnormal program termination. However, you can specify your own termination handler, if you like. You will need to refer to your compiler's documentation for directions on how this can be accomplished.
If you wish to rethrow an expression from within an exception handler, you can do so by simply calling **throw**, by itself with no exception. This causes the current exception to be passed on to an outer **try/catch** sequence.

EXAMPLES

1. The following program illustrates **catch(...)**:

```
// This example catches all exceptions.
#include <iostream>
using namespace std;

void Xhandler(int test)
{
    try
    {
        if(test==0)
            throw test; // throw int
        if(test==1)
            throw 'a'; // throw char
        if(test==2)
            throw 123.23; // throw double
    }
```

```
    catch(...) // catch all exceptions
    {
        cout << "Caught One!\n";
    }
}

int main()
{
    cout << "start\n";

    Xhandler(0);
    Xhandler(1);
    Xhandler(2);

    cout << "end";

    return 0;
}
```

This program displays the following output:

```
start
Caught One!
Caught One!
Caught One!
end
```

As you can see, all three **throws** were caught using the one **catch** statement.

2. One very good use for **catch(...)** is as the last **catch** of a cluster of catches. In this capacity it provides a useful default or "catch all" statement. For example, this slightly different version of the preceding program explicitly catches integer exceptions but relies upon **catch(...)** to catch all others:

```
// This example catch(...) as a default.
#include <iostream>
using namespace std;

void Xhandler(int test)
{
    try
    {
        if(test==0)
            throw test; // throw int
        if(test==1)
            throw 'a'; // throw char
        if(test==2)
            throw 123.23; // throw double
    }
    catch(int i) // catch an int exception
    {
        cout << "Caught " << i << '\n';
```

```
    }
    catch(...) // catch all other exceptions
    {
        cout << "Caught One!\n";
    }
}

int main()
{
    cout << "start\n";

    Xhandler(0);
    Xhandler(1);
    Xhandler(2);

    cout << "end";

    return 0;
}
```

The output produced by this program is shown here:

```
start
Caught 0
Caught One!
Caught One!
end
```

As this example suggest, using **catch(...)** as a default is a good way to catch all exceptions that you don't want to handle explicitly. Also, by catching all exceptions, you prevent an unhandled exception from causing an abnormal program termination.

3. The following program shows how to restrict the types of exceptions that can be thrown from a function:

```
// Restricting function throw types
#include <iostream>
using namespace std;

// This function can only throw ints, chars, and doubles.
void Xhandler(int test) throw(int, char, double)
{
    if(test==0)
        throw test; // throw int
    if(test==1)
        throw 'a'; // throw char
    if(test==2)
        throw 123.23; // throw double

}

int main()
```

```
{
     cout << "start\n";

     try
     {
          Xhandler(0); // also, try passing 1 and 2 to Xhandler
     }
     catch(int i)
     {
          cout << "Caught int\n";
     }
     catch(char c)
     {
          cout << "Caught char\n";
     }
     catch(double d)
     {
          cout << "Caught double\n";
     }

     cout << "end";

     return 0;
}
```

In this program, the function **Xhandler()** can throw only integer, character, and **double** exceptions. If it attempts to throw any other type of exception, an abnormal program termination will occur. (That is, **unexpected()** will be called.) To see an example of this, remove **int** from the list and retry the program.

It is important to understand that a function can only be restricted in what types of exceptions it throws back to the **try** block that called it. That is, a **try** block *within* a function can thrown any type of exception so long as it is caught *within* that function. The restriction applies only when throwing an exception out of the function.

4. The following change to **Xhandler()** prevents it from throwing any exceptions:

```
// This function can thrown NO exceptions!
void Xhandler(int test) throw()
{
     /*
        The following statements no longer work. Instead,
        they will cause an abnormal program termination.
     */
     if(test==0)
          throw test;
     if(test==0)
          throw 'a';
     if(test==2)
          throw 123.23;
}
```

As you have learned, you can rethrow an exception. The most likely reason for doing so is to allow multiple handlers access to the exception. For example, perhaps one exception

handler manages one aspect of an exception and a second handler copes with another. An exception can only be rethrown from within a **catch** block (or from any function called from within that block). When you rethrow an exception, it will not be recaught by the same **catch** statement. It will propagate to an outer **catch** statement. The following program illustrates rethrowing an exception. It rethrows a **char *** exception.

```
// Example of rethrowing an exception.
#include <iostream>
using namespace std;

void Xhandler()
{
    try
    {
        throw "hello"; // throw a const char *
    }
    catch(const char *) // catch a const char *
    {
        cout << "Caught const char * inside Xhandler\n";
        throw ; // rethrow const char * out of function
    }
}

int main()
{
    cout << "start\n";

    try
    {
        Xhandler();
    }
    catch(const char *)
    {
        cout << "Caught const char * inside main\n";
    }

    cout << "end";

    return 0;
}
```

This program displays the following output:

```
start
Caught const char * inside Xhandler
Caught const char * inside main
end
```

EXERCISES

1. Before continuing, compile and run all of the examples in this section. Be sure you understand why each program produces the output that it does.

2. What is wrong with this fragment?

```
try
{
     // ...
     throw 10;
}
catch(int *p)
{
     // ...
}
```

3. Show one way to fix the preceding fragment.

4. What **catch** expression catches all types of exceptions?

5. Here is a skeleton for a function called **divide()**.

```
double divide(double a, double b)
{
     // add error handling
     return a/b;
}
```

   This function returns the result of dividing **a** by **b**. Add error checking to this function using C++ exception handling. Specifically, prevent a divide-by-zero error. Demonstrate your solution in a program.

11.5 HANDLING EXCEPTIONS THROWN BY new

In Chapter 4 you learned that the modern specification for the **new** operator states that it will throw an exception if an allocation request fails. Since in Chapter 4 exceptions had not yet been discussed, a description of how to handle that exception was deferred until later. Now the time has come to examine precisely what occurs when **new** fails.

Before we begin, it is necessary to state that the material in this section describes the behavior of **new** as specified by Standard C++. As you should recall from Chapter 4, the precise action that **new** takes on failure has been changed several times since C++ was invented. Specifically, when C++ was first invented, **new** returned null on failure. Later this was changed such that **new** caused an exception on failure. Also, the name of this exception has changed over time. Finally, it was decided that a **new** failure will generate an exception by default, but that a null pointer could be returned instead, as an option. Thus, **new** has been implemented differently, at different times, by compiler manufacturers. Although all compilers will eventually implement **new** in compliance with Standard C++, not all currently do. If the code examples shown here do not work with your compiler, check your compiler's documentation for details on how it implements **new**.

In Standard C++, when an allocation request cannot be honored, **new** throws a **bad_alloc** exception. If you don't catch this exception, your program will be terminated, Although this behavior is fine for short sample programs, in real applications you must **catch** this exception and process it in some rational manner. To have access to this exception, you must include the header **<new>** in your program.

Note: *Originally this exception was called* ***xalloc****, and at the time of this writing many compilers still use the older name. However,* ***bad_alloc*** *is the name specified by Standard C++, and it is the name that will be used in the future.*

In Standard C++ it is also possible to have **new** return null instead of throwing an exception when an allocation failure occurs. This form of new is most useful when you are compiling older code with a modern C++ compiler. It is also valuable when you are replacing calls to **malloc()** with **new**. This form of **new** is shown here.

```
p_var=new(nothrow) type;
```

Here *p_var* is a pointer variable of *type*. The **nothrow** form of **new** works like the original version of **new** from years ago. Since it returns null on failure, it can be "dropped into" older code and you won't have to add exception handling. However, for new code, exceptions provide a better alternative.

EXAMPLES

1. Here is an example of **new** that uses a **try/catch** block to monitor for an allocation failure.

```
#include <iostream>
#include <new>
using namespace std;

int main()
{
    int *p;

    try
    {
        p = new int; // allocate memory for int
    }
    catch(bad_alloc xa)
    {
        cout << "Allocation failure.\n";
        return 1;
    }

    for(*p = 0; *p < 10; (*p)++)
        cout << *p << " ";

    delete p; // free the memory

    return 0;
}
```

   Her if an allocation failure occurs, it is caught by the **catch** statement.

2. Since the previous program is unlikely to fail under any normal circumstance, the following program demonstrates **new**'s exception-throwing capability bu forcing on allocation failure. It does this by allocating memory until it is exhausted.

```
// Force an allocation failure.
#include <iostream>
#include <new>
using namespace std;

int main()
```

```
{
    double *p;

    // this will eventually run out of memory
    do
    {
        try
        {
            p = new double[100000];
        }
        catch(bad_alloc xa)
        {
            cout << "Allocation failure.\n";
            return 1;
        }
    }
    while(p);

    return 0;
}
```

3. The following program shows how to use the **new(nothrow)** alternative. It reworks te preceding program and forces an allocation failure.

```
// Demonstrate the new(nothrow) alternative.
#include <iostream>
#include <new>
using namespace std;

int main()
{
    double *p;

    // this will eventually run out of memory
    do
    {
            p = new(nothrow) double[100000];
        if(p)
            cout << "Allocation OK\n";
        else
            cout << "Allocation Error.\n";
    }
    while(p);

    return 0;
}
```

As this program demonstrates, when you use the **nothrow** approach, you must check the pointer returned by **new** after each allocation request.

EXERCISES

1. Explain the difference between the behavior of **new** and **new(nothrow)** when an allocation failure occurs.

2. Given the following fragment, show two ways to convert it into modern C++-style code.

```
p =  malloc(sizeof(int));

if(!p)
{
     cout << "Allocation error.\n";
     exit(1);
}
```

SKILLS CHECK

Mastery Skills Check

At this point you should be able to perform the following exercises and answer the questions.

1. Create a generic function that returns the mode of an array of values. (The *mode* of a set is the value that occurs most often.)

2. Create a generic function that returns the summation of an array of values.

3. Create a generic bubble sort(or use nay other sorting algorithm you like).

4. Rework the **stack** class so that it can store pairs of different-type objects on the stack. Demonstrate your solution.

5. Show the general forms of **try**, **catch**, and **throw**. In your own words, describe their operation.

6. Again, rework the **stack** class so that stack over-and underflows are handled as exceptions.

7. Check your compiler's documentation. See whether it supports the **terminate()** and **unexpected()** functions. Generally, these functions can be configured to call any function you choose. If this is the case with your compiler, try creating your own set of customized termination functions that handle otherwise unhandled exceptions.

8. Thought question: Give a reason why having **new** generate an exception is a better approach than having **new** return null on failure.

Cumulative Skills Check

This section checks how well you have integrated material in this chapter with that from the preceding chapter.

1. In Chapter 6, Section 6.7, Example 3, a safe array class was shown. On your own, convert it into a generic safe array.

2. In Chapter 1, overloaded versions of the **abs()** function were created. As a better solution, create a generic **abs()** function on your own that will return the absolute value of any numeric object.